ASSIGNMENT 1

Apache Hive : Retail Transactions Analysis

Student Name	Niharika T R
USN	4GW23CI036
Branch	CSE(AI & ML)
Tool Covered	Apache Hive / HiveQL
Scenario	Retail transaction analysis
Date	1 October 2026

1. SCENARIO & PROBLEM STATEMENT

A retail company stores transaction records in CSV files on HDFS. The business intelligence team requires structured SQL access to analyze transaction revenue across product categories. The task is to create an external table in Hive mapping to the raw transaction data stored on HDFS, identify missing data, create a clean view, calculate transaction revenue, and analyze the valid transactions by product category.

2. ABOUT APACHE HIVE – THEORETICAL BACKGROUND

2.1 Apache Hive

Apache Hive is a data warehouse infrastructure built on top of Hadoop that provides a SQL-like interface, called HiveQL, for querying and managing large datasets stored in HDFS. Hive allows users familiar with SQL to work with big data without writing low-level MapReduce code directly.

2.2 Apache Hadoop

Apache Hadoop is an open-source framework that enables distributed storage and processing of large datasets. HDFS provides distributed storage, while YARN and processing engines support computation.

2.3 HDFS (Hadoop Distributed File System)

HDFS is a distributed, fault-tolerant file system designed to store large files. The NameNode manages filesystem metadata, while DataNodes store the actual data blocks.

3. ENVIRONMENT SETUP

The project was implemented in Ubuntu Linux using Apache Hadoop 3.3.6, HDFS, YARN/MapReduce, Apache Hive 4.0.1 and Beeline. HiveServer2 provides the JDBC endpoint used by Beeline.

```
beeline -u jdbc:hive2://localhost:10000
```

4. DATASET DESCRIPTION

The dataset used for this assignment is transactions.csv. It represents a small retail transaction log containing 12 records and four fields: transaction_id, product_category, quantity, and unit_price.

Column	Type	Description
transaction_id	STRING	Unique identifier for each transaction
product_category	STRING	Product category
quantity	INT	Number of units purchased
unit_price	DOUBLE	Price per unit

```
transaction_id,product_category,quantity,unit_price
T001,Electronics,2,50000
T002,Clothing,3,1200
T003,Electronics,1,30000
T004,Grocery,5,500
T005,Clothing,2,1500
T006,Electronics,1,45000
T007,Grocery,4,700
T008,Clothing,1,2000
T009,Electronics,2,25000
T010,Grocery,3,600
T011,Clothing,,1000
T012,,2,800
```

5. HDFS DATA STORAGE

The raw CSV file is stored in HDFS so that Hive can access it through an external table. The HDFS location used is /user/hive/transactions.

```
hdfs dfs -ls /user/hive/transactions
hdfs dfs -cat /user/hive/transactions/transactions.csv
```

6. EXTERNAL HIVE TABLE CREATION

A database named retail_db was used to logically group the Hive objects. An external table named transactions was created over the CSV data stored in HDFS. The external table keeps the underlying HDFS data separate from Hive metadata.

```
CREATE DATABASE IF NOT EXISTS retail_db;
USE retail_db;

CREATE EXTERNAL TABLE transactions (
    transaction_id STRING,
    product_category STRING,
    quantity INT,
    unit_price DOUBLE
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
STORED AS TEXTFILE
LOCATION '/user/hive/transactions'
TBLPROPERTIES ("skip.header.line.count"="1");
```

The table was verified using:

```
SHOW TABLES;
DESCRIBE FORMATTED transactions;
SELECT * FROM transactions;
```

7. MISSING DATA IDENTIFICATION

Before analysis, the dataset was checked for missing or blank values. The following query identifies rows where the product category is NULL or blank, or where quantity or unit price is NULL.

```
SELECT
    transaction_id,
    product_category,
    quantity,
    unit_price
FROM transactions
WHERE product_category IS NULL
   OR TRIM(product_category) = ''
   OR quantity IS NULL
   OR unit_price IS NULL;
```

The query identifies two incomplete records: T011 has a missing quantity and T012 has a blank product category. The unit price is present for both records.

8. DATA CLEANING

A view named clean_transactions was created to retain only records with a valid product category, quantity, and unit price.

```
CREATE VIEW clean_transactions AS
SELECT
    transaction_id,
    product_category,
    quantity,
    unit_price
FROM transactions
WHERE product_category IS NOT NULL
  AND TRIM(product_category) != ''
  AND quantity IS NOT NULL
  AND unit_price IS NOT NULL;

SELECT * FROM clean_transactions;
```

The original dataset contains 12 records. After excluding the two incomplete records, 10 valid transactions remain in the cleaned view.

9. TRANSACTION REVENUE

Revenue for each valid transaction is calculated by multiplying quantity by unit price.

```
SELECT
    transaction_id,
    product_category,
    quantity * unit_price AS revenue
FROM clean_transactions;
```

10. CATEGORY-WISE ANALYSIS

The valid records were examined category-wise using HiveQL filters.

```
SELECT * FROM clean_transactions
WHERE product_category = 'Electronics';

SELECT * FROM clean_transactions
WHERE product_category = 'Clothing';

SELECT * FROM clean_transactions
WHERE product_category = 'Grocery';
```

11. FINAL RESULTS

Category	Revenue	Quantity	Transactions	Average Order Value
Electronics	■225,000	6	4	■56,250.00
Clothing	■8,600	6	3	■2,866.67
Grocery	■7,100	12	3	■2,366.67
Overall	■240,700	24	10	■24,070.00

12. DATAFLOW

```
transactions.csv (HDFS)
        ↓
External Hive Table
        ↓
Missing / Blank Data Check
        ↓
clean_transactions View
        ↓
HiveQL Analysis
        ↓
Transaction Revenue
        ↓
Category-wise Analysis
        ↓
Final Results
```

13. CONCLUSION

This assignment provided hands-on experience with HDFS and Apache Hive for retail transaction analysis. The workflow covered storing raw CSV data in HDFS, creating an external Hive table, identifying incomplete records, creating a cleaned view, calculating transaction revenue, and analyzing the valid data by product category. The final analysis produced ■240,700 total revenue from 10 valid transactions.

EXECUTION SCREENSHOTS

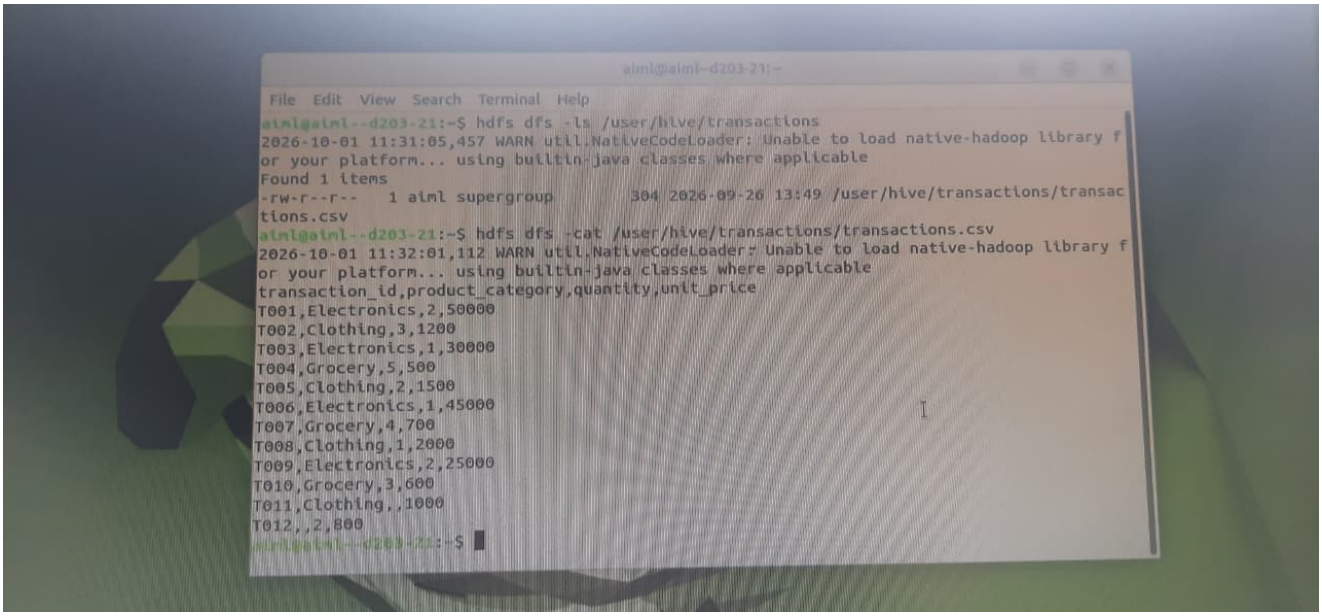


Figure 1: HDFS transaction file stored in /user/hive/transactions

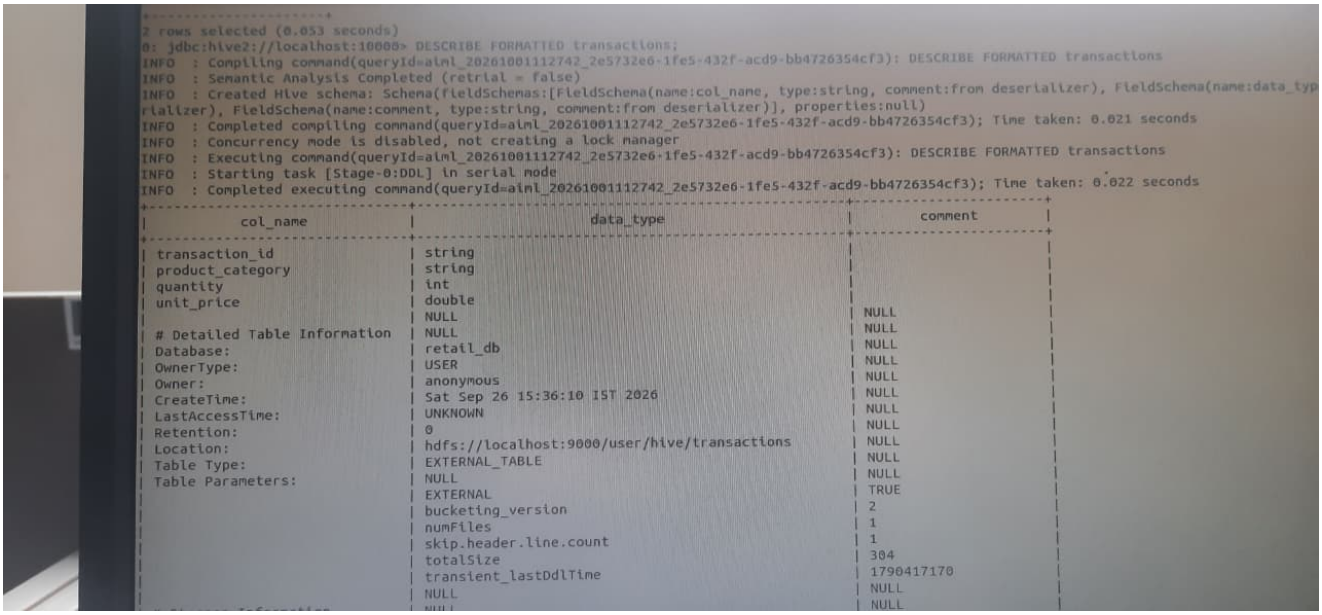
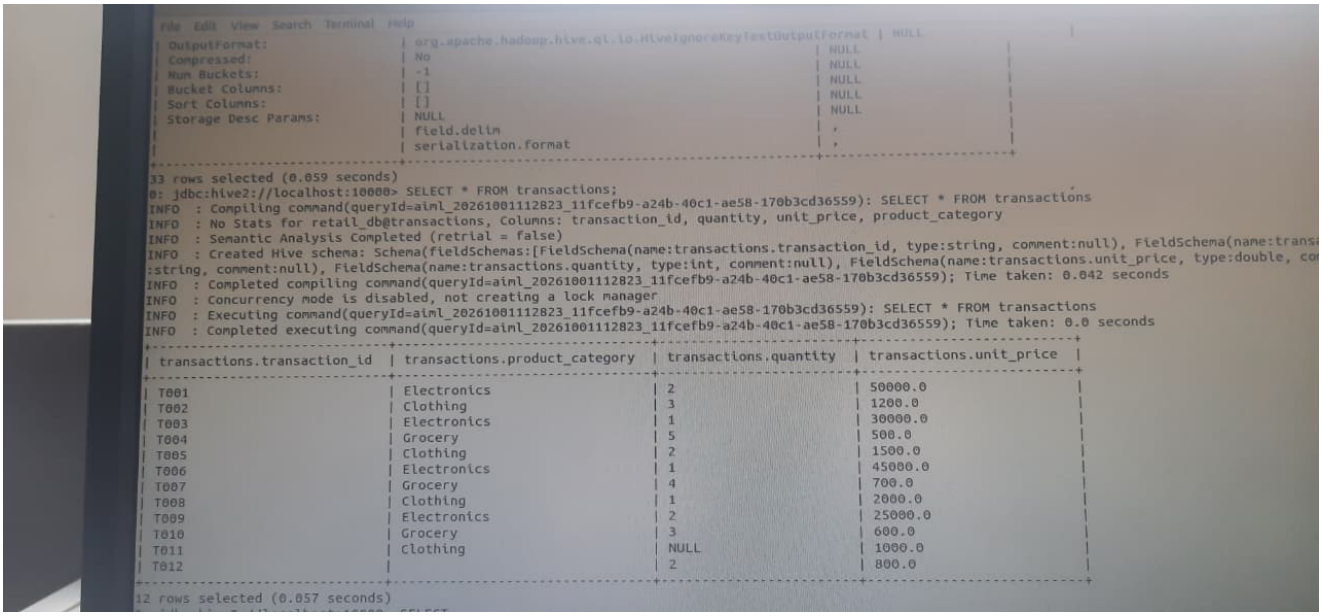


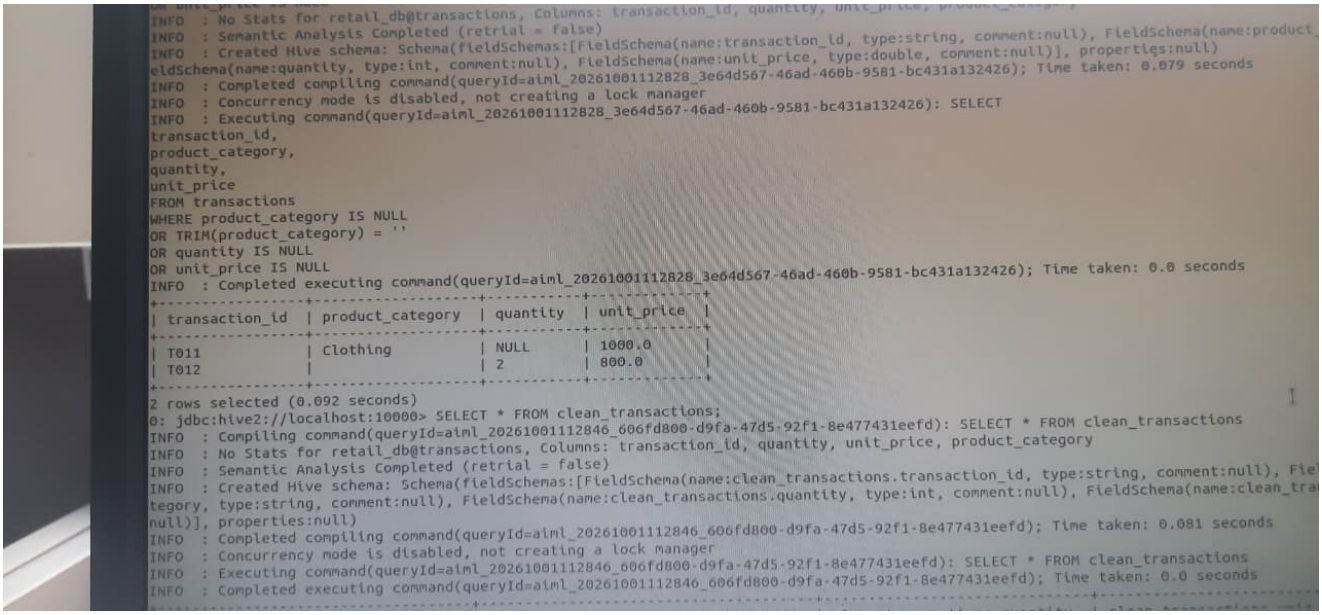
Figure 2: Hive table metadata using DESCRIBE FORMATTED

EXECUTION SCREENSHOTS



```
File  Edit  View  Search  Terminal  Help
| OutputFormat:                | org.apache.hadoop.hive.ql.io.HiveIgnoreKeyTextOutputFormat | NULL |
| Compressed:                  | No                                                         | NULL |
| Num Buckets:                 | -1                                                         | NULL |
| Bucket Columns:              | []                                                         | NULL |
| Sort Columns:                | []                                                         | NULL |
| Storage Desc Params:         | NULL                                                       | NULL |
|                              | field.delim                                                | ,    |
|                              | serialization.format                                       | ,    |
+------------------------------+------------------------------------------------------------+------+
33 rows selected (0.059 seconds)
0: jdbc:hive2://localhost:10000> SELECT * FROM transactions;
INFO  : Compiling command(queryId=aiml_20261001112823_11fcefb9-a24b-40c1-ae58-170b3cd36559): SELECT * FROM transactions
INFO  : No Stats for retail_db@transactions, Columns: transaction_id, quantity, unit_price, product_category
INFO  : Semantic Analysis Completed (retrial = false)
INFO  : Created Hive schema: Schema(fieldSchemas:[FieldSchema(name:transactions.transaction_id, type:string, comment:null), FieldSchema(name:trans
:string, comment:null), FieldSchema(name:transactions.quantity, type:int, comment:null), FieldSchema(name:transactions.unit_price, type:double, co
INFO  : Completed compiling command(queryId=aiml_20261001112823_11fcefb9-a24b-40c1-ae58-170b3cd36559); Time taken: 0.042 seconds
INFO  : Concurrency mode is disabled, not creating a lock manager
INFO  : Executing command(queryId=aiml_20261001112823_11fcefb9-a24b-40c1-ae58-170b3cd36559): SELECT * FROM transactions
INFO  : Completed executing command(queryId=aiml_20261001112823_11fcefb9-a24b-40c1-ae58-170b3cd36559); Time taken: 0.0 seconds
+-----------------------------+-------------------------------+-----------------------+-------------------------+
| transactions.transaction_id | transactions.product_category | transactions.quantity | transactions.unit_price |
+-----------------------------+-------------------------------+-----------------------+-------------------------+
| T001                        | Electronics                   | 2                     | 50000.0                 |
| T002                        | Clothing                      | 3                     | 1200.0                  |
| T003                        | Electronics                   | 1                     | 30000.0                 |
| T004                        | Grocery                       | 5                     | 500.0                   |
| T005                        | Clothing                      | 2                     | 1500.0                  |
| T006                        | Electronics                   | 1                     | 45000.0                 |
| T007                        | Grocery                       | 4                     | 700.0                   |
| T008                        | Clothing                      | 1                     | 2000.0                  |
| T009                        | Electronics                   | 2                     | 25000.0                 |
| T010                        | Grocery                       | 3                     | 600.0                   |
| T011                        | Clothing                      | NULL                  | 1000.0                  |
| T012                        |                               | 2                     | 800.0                   |
+-----------------------------+-------------------------------+-----------------------+-------------------------+
12 rows selected (0.057 seconds)
```

Figure 3: Original dataset displayed in Hive (12 transaction records)



```
INFO  : No Stats for retail_db@transactions, Columns: transaction_id, quantity, unit_price, product_category
INFO  : Semantic Analysis Completed (retrial = false)
INFO  : Created Hive schema: Schema(fieldSchemas:[FieldSchema(name:transaction_id, type:string, comment:null), FieldSchema(name:product_
eldSchema(name:quantity, type:int, comment:null), FieldSchema(name:unit_price, type:double, comment:null)], properties:null)
INFO  : Completed compiling command(queryId=aiml_20261001112828_3e64d567-46ad-460b-9581-bc431a132426); Time taken: 0.079 seconds
INFO  : Concurrency mode is disabled, not creating a lock manager
INFO  : Executing command(queryId=aiml_20261001112828_3e64d567-46ad-460b-9581-bc431a132426): SELECT
transaction_id,
product_category,
quantity,
unit_price
FROM transactions
WHERE product_category IS NULL
OR TRIM(product_category) = ''
OR quantity IS NULL
OR unit_price IS NULL
INFO  : Completed executing command(queryId=aiml_20261001112828_3e64d567-46ad-460b-9581-bc431a132426); Time taken: 0.0 seconds
+----------------+------------------+----------+-------------+
| transaction_id | product_category | quantity | unit_price  |
+----------------+------------------+----------+-------------+
| T011           | Clothing         | NULL     | 1000.0      |
| T012           |                  | 2        | 800.0       |
+----------------+------------------+----------+-------------+
2 rows selected (0.092 seconds)
0: jdbc:hive2://localhost:10000> SELECT * FROM clean_transactions;
INFO  : Compiling command(queryId=aiml_20261001112846_606fd800-d9fa-47d5-92f1-8e477431eefd): SELECT * FROM clean_transactions
INFO  : No Stats for retail_db@transactions, Columns: transaction_id, quantity, unit_price, product_category
INFO  : Semantic Analysis Completed (retrial = false)
INFO  : Created Hive schema: Schema(fieldSchemas:[FieldSchema(name:clean_transactions.transaction_id, type:string, comment:null), Fie
tegory, type:string, comment:null), FieldSchema(name:clean_transactions.quantity, type:int, comment:null), FieldSchema(name:clean_tra
null)], properties:null)
INFO  : Completed compiling command(queryId=aiml_20261001112846_606fd800-d9fa-47d5-92f1-8e477431eefd); Time taken: 0.081 seconds
INFO  : Concurrency mode is disabled, not creating a lock manager
INFO  : Executing command(queryId=aiml_20261001112846_606fd800-d9fa-47d5-92f1-8e477431eefd): SELECT * FROM clean_transactions
INFO  : Completed executing command(queryId=aiml_20261001112846_606fd800-d9fa-47d5-92f1-8e477431eefd); Time taken: 0.0 seconds
```

Figure 4: Missing records identified — T011 and T012

EXECUTION SCREENSHOTS

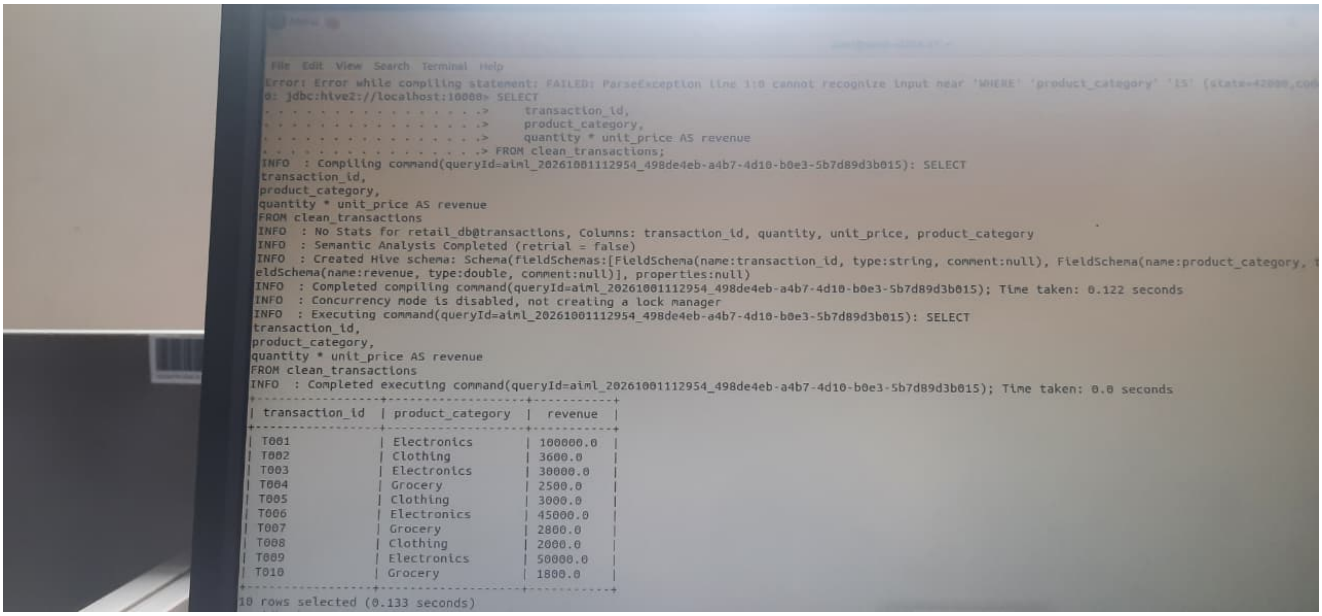


Figure 5: Clean transactions after removing incomplete records

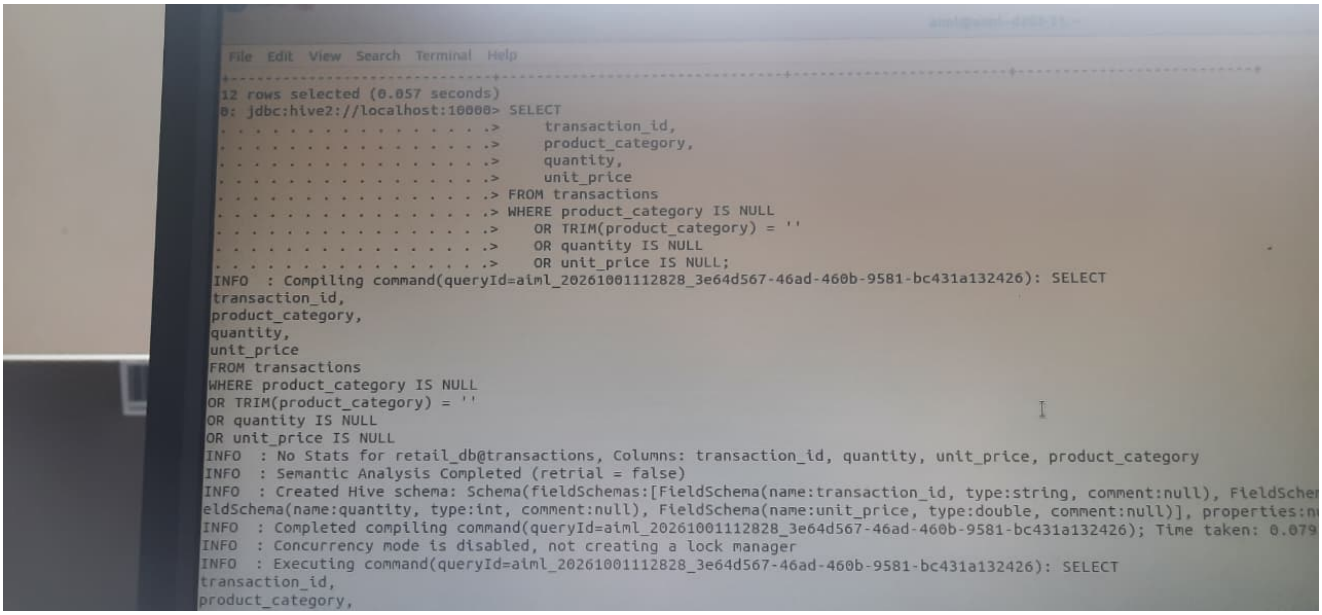


Figure 6: Missing-data query execution in Beeline